

Prueba de Aceptación 1

Objetivo: Verificar que la máquina arma y reconfigura correctamente sus símbolos y ruedas (posiciones fuera de rango, duplicados, símbolos inexistentes, giros hacia adelante y hacia atrás con vuelta completa, visibilidad) y que SlotMachineContest resuelve el problema de la maratón con cualquier tamaño de máquina de 2 a 8 ruedas, sin mostrarla en solve y mostrándola en simulate.

Parte A: Lógica de SlotMachine

1. Crear una máquina con `new SlotMachine()` y volverla visible con `makeVisible()`. Verificar que `isVisible()` es `true`.
2. Agregar cuatro símbolos: `addSymbol(1,"red")`, `addSymbol(2,"blue")`, `addSymbol(10,"green")` (debe ir al final) y `addSymbol(1,"yellow")` (debe ir al principio). Verificar que `symbols()` es `["yellow","red","blue","green"]`.
3. Intentar agregar un símbolo duplicado con `addSymbol(3,"red")`. Verificar que `ok()` es `false`, que `symbols()` sigue teniendo 4 elementos y que `lastMessage()` es "Accion no permitida: El color ya se encuentra entre las opciones". Cerrar el cuadro de diálogo.
4. Intentar eliminar un símbolo inexistente con `delSymbol("purple")`. Verificar que `ok()` es `false`, que `symbols()` sigue teniendo 4 elementos y que `lastMessage()` es "Accion no permitida: No se puede eliminar el símbolo purple porque no existe." Cerrar el cuadro de diálogo.
5. Agregar tres ruedas: `addWheel(1)`, `addWheel(2)` y `addWheel(0)` (debe insertarse al principio). Cada llamada muestra un mensaje (dos veces "Se agrego una rueda en la última posicion" y una vez "Se agrego una rueda en la primera posicion"); cerrarlos. Verificar que `configuration().length` es 3 y que `ok()` es `true`.
6. Colocar símbolos manualmente: `placeSymbol(1,"red")` y `placeSymbol(2,"blue")`. La rueda 3 queda con un símbolo aleatorio. Verificar que `configuration()[0]` es "red" y `configuration()[1]` es "blue". Si por casualidad las tres ruedas quedan iguales tras el primer `placeSymbol`, puede aparecer el mensaje de jackpot: se cierra y, al colocar la segunda rueda, la máquina vuelve a su color normal (gris).
7. Eliminar la rueda en una posición fuera de rango con `delWheel(10)`, lo que debe eliminar la última rueda. Verificar que `lastMessage()` es "Se elimino la ultima rueda", que `configuration().length` es 2, que `configuration()` es `["red","blue"]` y que `distinctSymbols()` es 2. Cerrar el cuadro de diálogo.
8. Intercambiar las ruedas con `swap(1,2)`. Verificar que `ok()` es `true` y que `configuration()` es `["blue","red"]`.
9. Ocultar la máquina con `makeInvisible()` y verificar que `isVisible()` es `false`. Ejecutar `spin(1,1)` y verificar que la máquina sigue invisible, que `ok()` es `true` y que `configuration()` es `["green","red"]` (la rueda 1 pasa de "blue" a "green").
10. Volver a mostrar la máquina con `makeVisible()` y verificar que `isVisible()` es `true`. Se debe ver el cuerpo gris con dos ruedas: una verde y una roja

11. Verificar la vuelta completa: spin(2,4) y luego spin(2,-4) dejan configuration() igual, ["green","red"] (la máquina tiene 4 símbolos). En ambos casos ok() es true y, al estar visible, el avance se muestra paso a paso en pantalla.
12. Verificar el giro hacia atrás: spin(2,-1)

Parte B: La solución del problema de la maratón

13. Crear un SlotMachineContest y ejecutar solve(4). Verificar que el resultado no es null, que tiene al menos una acción, que cada acción tiene 2 valores {rueda, pasos} y que la rueda está entre 1 y 4.
14. Obtener la máquina con ultimaMaquina(). Verificar que no es null, que isVisible() es false (solve no la muestra), que isJackpot() es true y que lastMessage() es "¡FELICIDADES HAS GANADO!" (el mensaje queda guardado aunque no se muestre).
15. Verificar que la máquina sigue abierta tras el jackpot (solo exit() la cierra): ok() es true, symbols().length es 4, configuration() tiene 4 elementos todos iguales entre sí y distinctSymbols() es 1.
16. Ejecutar solve(1), solve(0) y solve(-3). Verificar que devuelven 0 acciones
17. Ejecutar simulate(1) y verificar que aparece el mensaje "Con menos de dos ruedas no se puede hacer jackpot."

Prueba de Aceptación 2.

Objetivo: Demostrar una partida real y avanzada: un jugador que bloquea sus mejores ruedas para buscar el jackpot de forma estratégica, verificando la celebración automática (vía verificarJackpot()), el comportamiento de isJackpot() como consulta pura, y la reacción del sistema ante jugadas inválidas y ante el vaciado completo de la máquina.

Pasos:

1. Crear una máquina: SlotMachine machine = new SlotMachine();
2. Ocultarla con machine.makeInvisible() mientras se configura. Verifica que isVisible() es false
3. Crear tres símbolos válidos en orden: addSymbol(1,"blue"), addSymbol(2,"green"), addSymbol(3,"red"). Verifique que ok() es true en los tres casos y que symbols() es ["blue","green","red"].
4. Agregar cuatro ruedas consecutivas: addWheel(1), addWheel(2), addWheel(3), addWheel(4). Verifique que configuration().length es 4.
5. Forzar una configuración de jackpot parcial: spin(new String[]{"blue","blue","blue","red"}). Verifica que ok() es true
6. Verifique que isJackpot() retorna false (la cuarta rueda muestra un símbolo diferente) y que esta llamada no produce ningún efecto visible ni cambia lastMessage(), por ser ahora una consulta pura. Verifique que distinctSymbols() es 2 (solo "blue" y "red" están visibles)

7. Volverla visible con `makeVisible()` y verificar que `isVisible()` es `true`. Aquí sí se abre la ventana.
8. Bloquear las tres primeras ruedas: `lock(1)`, `lock(2)`, `lock(3)`. Verificar `ok()` en tres veces.
9. Ejecutar un giro general: `spin()`.
10. Verifique que las ruedas 1, 2 y 3 mantengan el símbolo "blue" intacto y que `ok()` es `true`.
11. Consultar `configuration()` para observar qué símbolo obtuvo la rueda 4 al azar, y guardar ese valor.
12. Colocar manualmente el símbolo faltante en la rueda 4: `placeSymbol(4,"blue")`. Esta llamada dispara internamente `verificarJackpot()`, así que si se completa el jackpot, la celebración visual (colores verde/amarillo y el mensaje) ocurre en este mismo paso, no en una consulta posterior.
13. Verifique que `isJackpot()` retorna `true` y que `lastMessage()` es exactamente "¡FELICIDADES HAS GANADO!". Verifique que `distinctSymbols()` es 1.
14. Volver a llamar `isJackpot()` una segunda vez y verificar que sigue retornando `true` sin volver a mostrar ningún mensaje ni repetir la celebración a diferencia de la versión anterior de esta prueba.
15. Desbloquear las ruedas 1 y 2: `unlock(1)`, `unlock(2)`.
16. Intentar un intercambio inválido cruzando la rueda 3 (aún bloqueada) con la rueda 4: `swap(3,4)`. Verifica que `ok()` es `false` y que `lastMessage()` es "Accion no permitida: Una de las ruedas esta bloqueada."
17. Rotar la rueda 1 (ya desbloqueada) 2 pasos: `spin(1,2)`, para sacarla del estado de jackpot.
18. Verifique que `isJackpot()` ahora retorna `false` y que, gracias a `verificarJackpot()`, la máquina restauró automáticamente sus colores normales (gris/rojo) al salir del jackpot, sin necesidad de llamar nada más.
19. Intente agregar un símbolo con un color no soportado: `addSymbol(1,"purple")`. Verifique que `ok()` es `false` y `lastMessage()` es "Accion no permitida: El color no esta disponible."
20. Eliminar todas las ruedas una a una con `delWheel(1)` repetidamente, hasta que `configuration().length` sea 0.
21. Intentar girar la máquina sin ruedas: `spin()`. Verifique que `ok()` es `false` y `lastMessage()` es "Accion no permitida: No se puede girar, porque esta vacia o no hay disponibles."
22. Cerrar definitivamente la máquina: `exit()`. Verifique que un intento posterior, por ejemplo `addWheel(1)`, no tiene ningún efecto (`configuration()` retorna `null` porque la máquina está cerrada) y que `lastMessage()` cambia a "Accion no permitida: la maquina fue cerrada con `exit()` y ya no se puede usar."

Parte B — La solución del problema de la maratón

Complementa la Parte B de la Prueba 1: aquí se cubre el tamaño máximo permitido, el límite de la construcción con `n` fuera de rango, el comportamiento visible de `simulate`, y el cierre de la máquina resultante.

1. Crear un `SlotMachineContest` y ejecutar `solve(10)`. Verificar que `ultimaMaquina().configuration().length` es 8, no 10 porque el constructor `SlotMachine(int n)` limita `n` a un máximo de 8 símbolos/ruedas disponibles.
2. Verificar que ninguna acción devuelta por `solve(10)` referencia una rueda 9 o 10 todas deben estar entre 1 y 8, confirmando que el algoritmo trabajó sobre la máquina realmente creada (de 8 ruedas), no sobre el `n` original solicitado.
3. Verificar que `ultimaMaquina().isJackpot()` es `true` y que `distinctSymbols()` es 1, confirmando que el algoritmo también gana correctamente en el tamaño máximo permitido.
4. Ejecutar `simulate(5)` y observarlo en pantalla. Al finalizar, verificar que `ultimaMaquina().isVisible()` es `true` (a diferencia de `solve`, aquí la máquina sí queda visible) y que `ultimaMaquina().isJackpot()` es `true`.
5. Cerrar esa máquina con `ultimaMaquina().exit()`. Intentar luego `ultimaMaquina().spin(1,1)` y verificar que no tiene ningún efecto: `ok()` es `false` y `lastMessage()` es "Accion no permitida: la maquina fue cerrada con exit() y ya no se puede usar."
6. Ejecutar `solve(2)` cinco veces seguidas (en un ciclo), verificando en cada una que `contest.ultimaMaquina().isJackpot()` es `true`, confirmando que el algoritmo también es consistente en el tamaño mínimo válido (`n=2`), pese a que la posición inicial aleatoria cambia en cada llamada.